

Does the event-behaviour edge actually exist?

A full audit of the buy/sell backtest — how its rules are generated, every place look-ahead and bias creep in, and what survives once the trades are made honest. Plus the data pipeline, the fundamentals top-up to a complete Nifty 50, and the dashboard change.

Scope: Nifty 50 • Data through Jul 2026 • Prices from Yahoo, events from NSE, fundamentals from Screener

01 Bottom line

No tradeable event edge survives honest testing on the Nifty 50. Every apparently-profitable rule either (a) requires buying *before* news nobody could have known was coming, (b) sits on a data bug, or (c) fails to beat buying on random dates once the market is stripped out and the rule is tested on events it never saw.

0

rules that beat random dates out-of-sample with $t > 2$ in a way that replicates

~40%

of announcement dates were silently corrupted by a date-parsing bug

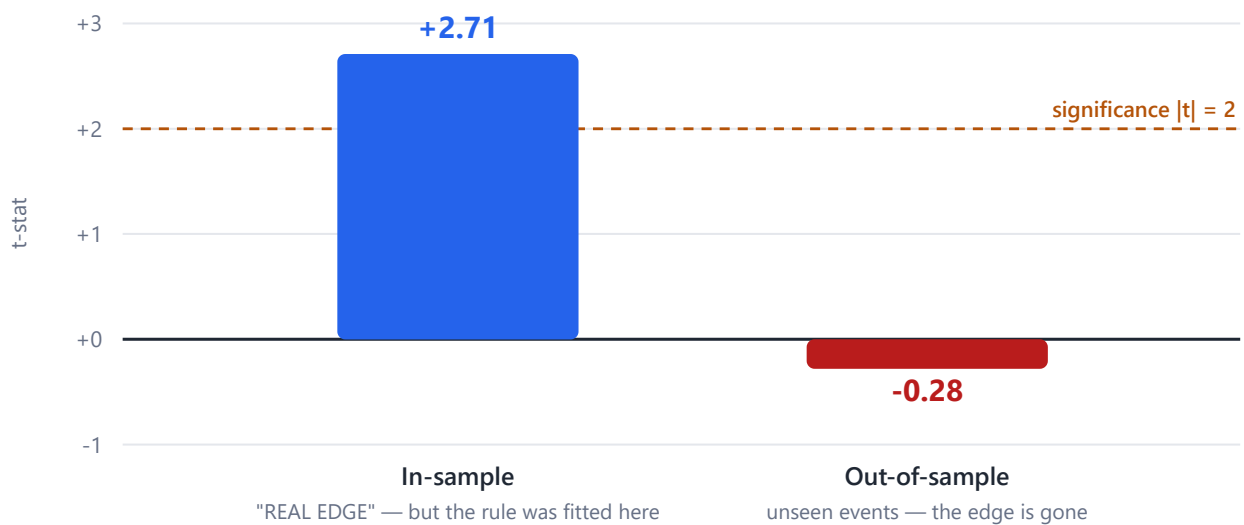
~80%

of the announcement "edge" lived in the un-tradeable pre-news window

52/52

Nifty 50 stocks now have complete fundamentals (was 49)

The announcement "edge" evaporates out-of-sample



Month-clustered t-statistic for the best ANNOUNCEMENT rule. In-sample it clears the $|t|=2$ bar and prints "REAL EDGE"; on events it never saw, it is indistinguishable from zero. The whole edge was curve-fitting.

The statistical scaffolding in the backtest (train/test split, market-adjustment, month-clustered t-stats) is genuinely careful work. The problem is *upstream* of the statistics: the events aren't tradeable in advance and the universe is survivorship-selected. No amount of clean t-testing rescues a signal you couldn't have acted on.

02 The pipeline & scripts

Everything is driven off two data sources — daily **adjusted prices** from Yahoo (cached in `processed/price_cache/`) and **corporate event feeds** from NSE (per-stock `<SYMBOL>/*.csv`), plus **Screener fundamentals** in five folders.

SCRIPT	WHAT IT DOES	OUTPUT
<code>event_behaviour.py</code>	Grid-searches buy T-N / sell T+M for every event, aggregates per event type, picks the "best" rule	<code>event_behaviour_summary.csv</code>
<code>nifty_backtest.py</code>	Honest version: train/test split, market-adjusted, month-clustered t-stats, verdict	<code>nifty_backtest.csv</code>
<code>optimal_rules.py</code> new	Per-stock optimal rule, two passes (descriptive vs tradeable), train/test + random-date placebo	<code>optimal_rules.csv</code>
<code>fundamental_buckets.py</code> new	Pools all stocks, buckets events by the company's fundamentals <i>at the time</i> , finds a rule per bucket	<code>fundamental_buckets.csv</code>
<code>earnings_surprise.py</code> new	Post-earnings drift (PEAD): do quarterly beats drift up and misses drift down?	<code>earnings_surprise.csv</code>
<code>download_fundamentals.py</code> new	Scrapes Screener statements + schedules API into the exact folder CSV format	5 folders × CSV
<code>stock_server.py</code> / <code>nifty_dash.py</code>	The web dashboard (Fundamentals, Behaviour, Financial Results tabs)	localhost:8090

The five fundamentals folders — `pn1` , `balance_sheet` , `cash_flow` , `ratios` (annual, Mar-2015→Mar-2026) and `quarterly` (Mar-2023→Mar-2026) — hold one CSV per stock, Screener format.

03 How the buy/sell rules are made — not hardcoded

The "buy N days before / sell M days after" recommendation is **computed by brute-force grid search**, not written into the code. For every event it prices all 8×8 combinations of entry and exit day, averages each cell across every event of that type, and picks the winning cell. Nothing about "buy 7 days before" is a constant; it falls out of the data.

Three things *are* effectively hardcoded — the ± 8 -day search window (`N_BEFORE = N_AFTER = 8` , so the rule can never point outside that range), a drill-in report frozen at "6 days before / 3 days after", and a selection rule that quietly disagrees between files (highest win-rate vs highest average return).

04 The honesty rails

Every new analysis applies the same defences, so a good-looking number has to survive all of them:

Train / test split

Fit the rule on the oldest 70% of events, report only on the newest 30% it never saw.

Market-adjusted

Subtract the NIFTY's move over the exact same days. If the edge vanishes, it was just market drift.

Random-date placebo

Same holding period on random dates. A rule that can't beat random days isn't a rule.

Month-clustered t

Trades on 50 stocks share market days — the naive t is 3-5× inflated. Cluster by calendar month.

Knowability constraint

Entry is forbidden before the event was *public* (intimation date for results, disclosure time for news).

Costs

0.10% round-trip charged on every trade (still optimistic vs real ~0.25% delivery cost).

05 Look-ahead & biases found

① The date-parsing bug critical

Announcements are dated from `sort_date`, which is ISO (`2026-07-10`), but parsed with `dayfirst=True`. So **10 July silently becomes 7 October** — every announcement whose day-of-month is ≤ 12 (roughly 40%) is priced against the wrong days. The other feeds use month-name dates and are safe. This alone invalidates the announcement results, which were 77% of the test trades.

② You can't buy before an unscheduled announcement critical

Press releases, acquisitions, credit ratings — nobody knows these are coming. The optimizer's rule "buy 8 days before the announcement" is physically impossible to place. When entry is *allowed* to cheat, the optimizer cheats **81% of the time** (44 of 54 stocks put entry before the event).

③ Results entry lands before the date was public

Board-meeting/results dates are only knowable from the *intimation*, which the feeds show arrives ~ 7 calendar days (≈ 5 trading days) ahead. The fitted rules buy at T-7 / T-8 — *before* the intimation was filed.

④ More biases

BIAS	EFFECT
Survivorship	Universe is <i>today's</i> Nifty 50 back to 2007 — every stock that fell out for underperforming is absent. Market-adjustment doesn't fix it.
Corner solutions	Winning rules sit at the grid boundary (8/8, 7/8...). A real event effect peaks at an interior cell; a boundary hit means it's harvesting drift, not the event.
Non-independent events	One quarterly result spawns a BOARD_MEETING + ANNOUNCEMENT + RESULTS row within days — the same trade counted three times.
Leaky split	Train/test cut was computed <i>per symbol</i> , so 2023 test events sit beside other stocks' 2023 train events; ~ 0.8 correlation leaks the fit.
Adjusted close	Yahoo adjclose back-adjusts dividends out, so the ex-date drop doesn't exist in the series used.

06 Does any edge survive?

Reliance, honestly tested (19.5 years)

Results, annual results and dividends were each tested with entry forbidden before the event was public, net of cost, market-adjusted, against a placebo.

EVENT	TRADES	VS NIFTY	T	WIN%	PLACEBO	VERDICT
Quarterly results	56	+0.10%	+0.18	42%	+0.08%	no edge
Annual results	16	-0.10%	-0.08	40%	+0.27%	no edge
Dividends (full)	16	+1.03%	+1.56	75%	+0.35%	died post-2018

The dividend "edge" was **+2.86% (7/7 wins) in 2011–2017 and -0.40% in 2018–2026** — gone for the last nine years, and a single COVID-rebound trade (+13.8%, Apr-2020) carries much of the historical average.

The one genuinely interesting shape: Reliance *drifts up into* its results (+0.8% vs market over the 5 days before) and *sells off after* (-0.6% in the two days after) — classic "buy the rumour, sell the news". The optimizer independently landed on **buy T-5, sell T-1** (sell before the result is out). It's inside the knowable window, so it's legal — but only 16 events, $t \approx 1.8$, unproven.

Reliance drifts up into results, then sells off — “buy the rumour, sell the news”



Average market-adjusted move around 56 quarterly results, measured from the event-day close. The stock climbs into the announcement and gives it back after. The only legal, profitable window is buy T-5 / sell T-1 — before the news — though at 16 net events it is unproven.

The dividend ex-date "discount recovery"

Tested on **raw** prices (adjclose hides the drop) with split-corrected dividend amounts — after catching a bug where the raw historical ₹ dividend was added onto a 8×-split-adjusted price.

There is **no discount to recover**. Reliance's dividend is ~0.4% of price; its daily volatility is ~1.5%. The stock drops **0.03× its dividend** on the ex-date — statistically zero — and the median recovery is **0 days** because half the time there was never a dip. The signal is 4× smaller than the noise it hides in.

This trade could exist in high-yield names (COALINDIA ~6%, ONGC/POWERGRID/ITC ~3-5%) where the discount is large relative to daily noise — not in a 0.4%-yield growth stock.

Per-stock optimal rules (55 stocks)

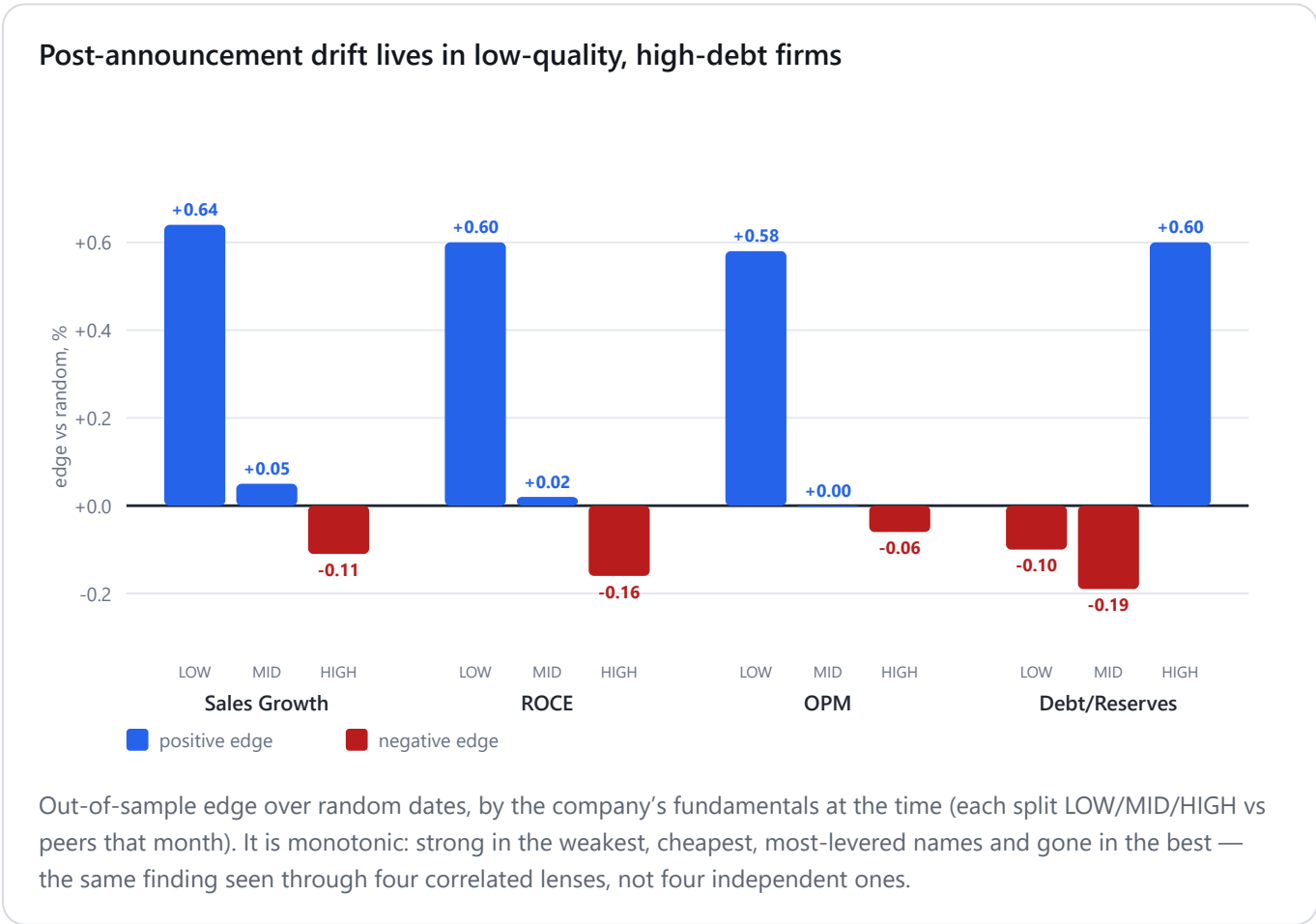
"Beat random dates" landed at 26–30 of ~49 stocks — a coin flip. Forcing entry after the news, the average announcement edge collapsed to **+0.05%** (zero). Only 2 of 53 cleared $t > 2$ in tradeable mode — exactly the false-positive rate expected from testing 53 stocks.

Fundamental buckets — the most interesting negative

Results: nothing in any bucket. **Announcements:** a clean, monotonic pattern — post-news drift exists in *low-quality, low-margin, high-debt* companies and is absent in high-quality ones.

DIMENSION (EDGE OVER RANDOM)	LOW	MID	HIGH
Sales Growth %	+0.64%	+0.05%	-0.11%
ROCE %	+0.60%	+0.02%	-0.16%
OPM %	+0.58%	+0.00%	-0.06%
Debt / Reserves	-0.10%	-0.19%	+0.60%

Strongest cell: high-leverage, buy T+0 / sell T+12 → +0.85% vs market, t(month)=3.01, n=829. but the four dimensions select the *same* stocks (one finding, not four), it's a corner solution (always sell T+12), and ~0.6% barely clears realistic costs. Not a green light — a single hypothesis worth a targeted test.



Earnings surprise / PEAD (quarterly, 2023–2026)

No post-earnings drift in the Nifty 50. The market-neutral long-beat/short-miss spread is flat-to-negative (the one "significant" cell is *negative*, $t = -2.11$ — noise). Beats don't even pop on day 1. The 50 most liquid, most-covered stocks in India price results efficiently within the day.

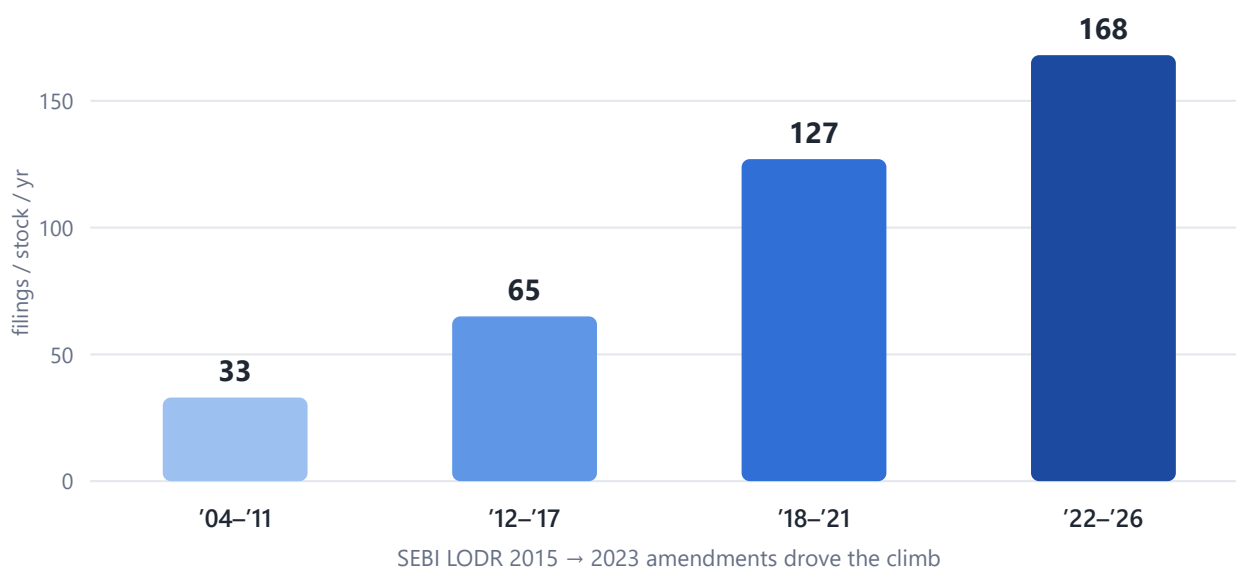
PEAD lives in small, neglected stocks and down markets — the opposite of a Nifty-50 bull-run sample. Absence here doesn't prove absence everywhere.

07 Data coverage & the regime change

Reliance has **19.5 years of prices** (2007→2026) and 21.6 years of announcements. But the announcement count is deeply skewed by regulation, not reality — **56% of all announcements are from 2020 onward**, and per-stock filings grew $\sim 5\times$ (33 → 168 per year).

ERA	FILINGS / STOCK / YR	WHAT ACTUALLY CHANGED
2004–2011	33	baseline
2015→2016 jump	65	SEBI LODR 2015 (Reg 30 / Schedule III enumerates disclosable events)
2018–2021	127	mandatory "Loss of Share Certificates", newspaper copies, etc. appear
2022→2024 jump	168	LODR 2nd Amendment 2023 — 12-hour deadlines, disclosure regardless of materiality

Announcement volume grew $\sim 5\times$ — regulation, not more news



Per-stock disclosures per year by era. "Press Release" (real company news) stayed flat at $\sim 13/\text{yr}$; the growth is new mandatory categories. A chronological train/test split on announcements therefore trains and tests on two different disclosure regimes.

The growth is *more categories*, not more news — "Press Release" (real company news) plateaued at $\sim 13/\text{yr}$ while whole new administrative categories appeared from zero. A chronological train/test split on announcements therefore trains on a pre-2016, 33-filings world and tests on a post-2023,

200-filings world — **two different populations**, which is exactly why fitted announcement rules don't transfer.

Verified against SEBI's LODR 2015 and the June-2023 Second Amendment. Two unexplained items flagged: 2025 announcement volume drops 28% (possible feed gap), and Reliance's results feed truncates at Jan-2025.

08 Fundamentals top-up — now 52/52

Three Nifty names were missing from all five folders. Built `download_fundamentals.py` to scrape Screener's public company page plus its `/api/company/<id>/schedules/` endpoint, reproducing the exact folder format (validated to **zero value differences** against the existing Reliance files before writing anything).

STOCK	SCREENER SOURCE	FY26 SANITY CHECK	CAVEAT
LTIM	code 540005 ("LTM Ltd")	Sales ₹42,308 cr · EPS 169 · ROCE 30%	—
SBILIFE	SBILIFE (standalone)	Premium ₹1.12 L cr · EPS 24.6	insurer — no growth% rows
TATAMOTORS	TMPV (id 3370)	Sales ₹3.36 L cr (full JLR-era group)	FY26 distorted by 2025 demerger

Fixing this exposed why care was needed: the schedule API uses an internal warehouse id (not the URL slug), insurers leave the consolidated P&L empty, and Tata's 2025 demerger split the history onto the TMPV entity while a 2-quarter stub (TMCV) carries the "Tata Motors Ltd" name.

09 Dashboard change — offline Excel export

The Financial Results "Download Excel" used to fetch the raw XBRL filing online. It now builds the workbook straight from the local folders — a new `/api/statements_excel` endpoint and `build_statements_excel()` in `stock_server.py`. One click → `<SYMBOL>_financials.xlsx` with five sheets (P&L · Balance Sheet · Cash Flow · Ratios · Quarterly), fully offline. Works for all 52 Nifty stocks.

live on :8090

offline / no online fetch

uncommitted working-tree code

10 Open questions & what's next

1. **Fix the date bug** in `event_behaviour.py` — switch the announcements feed from `sort_date` to `an_dt`. It mis-dates ~40% of announcements across all 50 stocks regardless of the strategy question.
2. **Test the two live hypotheses** that aren't yet dismissed: (a) Reliance's buy-T5/sell-T1 pre-result drift, across all 50 stocks for a real sample; (b) the high-leverage post-announcement drift — extend the exit past T+12 to see if it plateaus (real drift) or keeps rising (just leveraged beta).
3. **Dividend capture on high-yield names** (COALINDIA, ONGC, POWERGRID) where the discount is big enough to measure.
4. **PEAD on small/mid-caps** — 3,400 stocks have fundamentals; the anomaly has room to exist where coverage is thin.
5. **Clamp the dashboard Behaviour tab to tradeable mode** so it stops surfacing rules that buy before un-knowable news.

Generated documentation of the investigation to date · Behaviour Analysis project · open this file in any browser. Figures are point-in-time from the runs described above; re-run the scripts to refresh.